

Python Programming

Day 5: 클래스와 객체지향 프로그래밍 기초

Class, Object, Constructor, Method, self

Contents

1	학습 목표	2
2	클래스와 객체	2
2.1	클래스 정의	2
2.2	객체 생성	3
2.3	최종 코드: 01_first_class.py	3
3	생성자와 인스턴스 변수	3
3.1	생성자의 기본 구조	3
3.2	매개변수와 인스턴스 변수	4
3.3	최종 코드: 02_constructor.py	4
4	인스턴스 메서드	4
4.1	메서드 정의와 호출	5
4.2	메서드로 객체 상태 변경하기	5
4.3	최종 코드: 03_instance_methods.py	5
5	self의 의미	6
5.1	메서드 호출의 내부 동작	6
5.2	self는 관례적인 이름	6
5.3	C++의 this와 비교	6
5.4	최종 코드: 04_self.py	7
6	여러 객체와 객체의 독립성	7
6.1	한 객체만 변경하기	7
6.2	같은 클래스와 서로 다른 객체	8
6.3	최종 코드: 05_multiple_objects.py	8
7	Day 5 종합 실습: 학생 관리 프로그램	8
7.1	프로그램 요구사항	8
7.2	최종 코드: practice_day05.py	9
7.3	실행 결과	9
7.4	프로그램 구조 분석	10
8	실습 파일 목록	10
9	핵심 요약	11

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 클래스와 객체의 차이를 설명하기
- `class` 문으로 새로운 클래스를 정의하기
- 클래스로부터 객체를 생성하기
- `__init__()` 생성자로 객체의 초기 상태 설정하기
- 인스턴스 변수에 데이터를 저장하고 접근하기
- 인스턴스 메서드를 정의하고 호출하기
- `self`가 현재 객체를 가리키는 원리 설명하기
- 같은 클래스로 만든 여러 객체가 서로 독립적임을 확인하기
- 메서드를 사용하여 객체의 상태를 변경하기
- 클래스, 생성자, 메서드, 여러 객체를 결합한 학생 관리 프로그램 구현하기

2. 클래스와 객체

지금까지는 변수와 함수를 각각 따로 정의하여 프로그램을 작성했다. 그러나 프로그램이 커지면 서로 관련된 데이터와 기능을 한곳에 묶어 관리하는 편이 더 효율적이다. 객체지향 프로그래밍(object-oriented programming)은 데이터를 나타내는 속성과 그 데이터를 처리하는 동작을 하나의 객체로 구성하는 방식이다.

클래스(class)는 객체를 만들기 위한 설계도이고, 객체(object)는 그 설계도로부터 실제로 생성된 값이다. 객체를 인스턴스(instance)라고도 부른다.

용어	의미
클래스	객체를 만들기 위한 설계도
객체 또는 인스턴스	클래스로부터 생성된 실제 값
속성	객체가 저장하는 데이터
메서드	객체가 수행하는 동작

예를 들어 `Student` 클래스는 학생을 표현하는 설계도이다. 이 클래스로부터 `Kim`, `Lee`, `Park`과 같은 서로 다른 학생 객체를 만들 수 있다.

2.1. 클래스 정의

Python에서는 `class` 키워드로 클래스를 정의한다.

```
1 class Student:
2     pass
```

`pass`는 문법적으로 코드 블록이 필요하지만 아직 실행할 내용이 없을 때 사용하는 문장이다. 위 클래스는 아직 속성과 메서드가 없는 빈 클래스이다.

2.2. 객체 생성

클래스 이름 뒤에 괄호를 붙이면 객체가 생성된다.

```
1 student = Student()
```

이제 `student`는 `Student` 클래스로부터 만들어진 객체를 가리킨다. 객체를 출력하면 클래스 이름과 객체의 메모리 위치를 포함한 정보가 나타난다.

```
1 print(student)
```

출력 예시는 다음과 같다.

```
<__main__.Student object at 0x000001F4A6E3B6A0>
```

메모리 주소는 실행할 때마다 달라질 수 있다.

2.3. 최종 코드: 01_first_class.py

```
1 class Student:
2     pass
3
4
5 student = Student()
6
7 print(student)
```

실습

`Car`라는 빈 클래스를 정의하고 `car = Car()`로 객체를 생성한 뒤 출력한다. 출력된 정보에서 클래스 이름을 확인한다.

핵심 포인트

클래스는 하나의 새로운 자료형을 정의한다. `Student()`를 호출할 때마다 해당 자료형의 새로운 객체가 생성된다.

3. 생성자와 인스턴스 변수

빈 객체만으로는 학생의 이름이나 나이를 저장할 수 없다. 객체가 생성될 때 필요한 값을 전달받아 초기 상태를 설정하려면 생성자(constructor)를 사용한다.

Python에서는 `__init__()` 메서드가 객체 생성 직후 자동으로 호출된다. 일반적으로 이 메서드 안에서 객체가 가져야 할 인스턴스 변수를 만든다.

3.1. 생성자의 기본 구조

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
```

객체를 만들 때 생성자에 필요한 값을 전달한다.

```
1 student = Student("Kim", 20)
```

이 호출에서는 문자열 `"Kim"`이 매개변수 `name`으로, 정수 `20`이 매개변수 `age`로 전달된다.

3.2. 매개변수와 인스턴스 변수

다음 문장은 이름이 비슷한 두 변수를 사용한다.

```
1 self.name = name
```

오른쪽의 `name`은 생성자로 전달받은 매개변수이다. 왼쪽의 `self.name`은 현재 객체 안에 저장되는 인스턴스 변수이다.

표현	의미
<code>name</code>	생성자가 전달받은 지역 매개변수
<code>self.name</code>	객체에 저장되는 인스턴스 변수

객체가 생성된 후에는 점 연산자로 인스턴스 변수에 접근한다.

```
1 print(student.name)
2 print(student.age)
```

3.3. 최종 코드: 02_constructor.py

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6
7 student = Student("Kim", 20)
8
9 print(student.name)
10 print(student.age)
```

실행 결과는 다음과 같다.

```
Kim
20
```

실습

Book 클래스를 만들고 생성자에서 `title`과 `author`를 저장한다. `Book("Python", "Lee")` 객체를 생성한 뒤 두 속성을 출력한다.

핵심 포인트

생성자는 객체를 만드는 함수 자체가 아니라, 객체가 생성된 직후 그 객체의 초기 상태를 설정하는 특별한 메서드이다.

4. 인스턴스 메서드

클래스 안에 정의되어 객체가 사용하는 함수를 메서드(method)라고 한다. 인스턴스 메서드는 객체의 속성을 읽거나 변경하며, 객체가 수행할 동작을 표현한다.

4.1. 메서드 정의와 호출

```

1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def introduce(self):
7         print(f"My name is {self.name}.")
8         print(f"I am {self.age} years old.")

```

객체의 메서드는 점 연산자로 호출한다.

```

1 student = Student("Kim", 20)
2 student.introduce()

```

실행 결과는 다음과 같다.

```

My name is Kim.
I am 20 years old.

```

4.2. 메서드로 객체 상태 변경하기

메서드는 객체의 속성을 출력하는 것뿐 아니라 값을 변경할 수도 있다.

```

1 def birthday(self):
2     self.age += 1
3     print(f"{self.name} is now {self.age} years old.")

```

birthday()를 호출하면 현재 객체의 age가 1 증가한다. 변경된 값은 메서드 호출이 끝난 뒤에도 객체 안에 유지된다.

4.3. 최종 코드: 03_instance_methods.py

```

1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def introduce(self):
7         print(f"My name is {self.name}.")
8         print(f"I am {self.age} years old.")
9
10    def birthday(self):
11        self.age += 1
12        print(f"{self.name} is now {self.age} years old.")
13
14
15 student = Student("Kim", 20)
16
17 student.introduce()
18 student.birthday()
19 student.introduce()

```

실행 결과는 다음과 같다.

```

My name is Kim.
I am 20 years old.
Kim is now 21 years old.
My name is Kim.
I am 21 years old.

```

실습

Book 클래스에 `show_info()` 메서드를 추가한다. 메서드를 호출하면 제목과 저자가 각각 `Title: ...`, `Author: ...` 형식으로 출력되도록 작성한다.

5. self의 의미

`self`는 현재 메서드를 호출한 객체 자신을 가리킨다. 같은 메서드를 여러 객체가 공유할 수 있는 이유는 호출한 객체가 첫 번째 인수로 자동 전달되기 때문이다.

5.1. 메서드 호출의 내부 동작

다음 객체 두 개를 생각해보자.

```
1 student1 = Student("Kim")
2 student2 = Student("Lee")
```

다음 호출은 서로 다른 객체를 사용한다.

```
1 student1.introduce()
2 student2.introduce()
```

개념적으로 Python은 첫 번째 호출을 다음과 비슷하게 처리한다.

```
1 Student.introduce(student1)
```

따라서 첫 번째 호출에서 `self`는 `student1`을 가리킨다. 두 번째 호출은 다음과 비슷하다.

```
1 Student.introduce(student2)
```

이때 `self`는 `student2`를 가리킨다. 그러므로 같은 `self.name` 코드가 호출한 객체에 따라 서로 다른 이름을 읽는다.

5.2. self는 관례적인 이름

문법적으로 첫 번째 매개변수의 이름을 반드시 `self`라고 해야 하는 것은 아니다.

```
1 class Student:
2     def introduce(me):
3         print(me.name)
```

그러나 Python 코드에서는 첫 번째 인스턴스 매개변수 이름으로 `self`를 사용하는 것이 확립된 관례이다. 다른 이름을 사용하면 코드를 읽는 사람이 혼란을 느낄 수 있으므로 항상 `self`를 사용한다.

5.3. C++의 this와 비교

Python	C++
<code>self</code>	<code>this</code>
<code>self.name</code>	<code>this->name</code>
명시적인 첫 번째 매개변수	자동으로 존재하는 포인터

두 언어 모두 현재 메서드를 실행하는 객체를 구분하는 역할을 한다. Python은 메서드 정의에 `self`를 직접 적지만, 메서드를 호출할 때는 객체를 별도로 전달하지 않는다.

5.4. 최종 코드: 04_self.py

```

1 class Student:
2     def __init__(self, name):
3         self.name = name
4
5     def introduce(self):
6         print(f"My name is {self.name}.")
7
8
9 student1 = Student("Kim")
10 student2 = Student("Lee")
11
12 student1.introduce()
13 student2.introduce()

```

실행 결과는 다음과 같다.

```

My name is Kim.
My name is Lee.

```

실습

Car 클래스에 brand 속성과 show.brand() 메서드를 만든다. Hyundai와 Toyota 객체를 각각 생성하여 동일한 메서드가 서로 다른 속성을 출력하는지 확인한다.

핵심 포인트

object.method() 형태로 호출하면 Python이 object를 메서드의 첫 번째 매개변수인 self에 자동으로 전달한다.

6. 여러 객체와 객체의 독립성

하나의 클래스에서는 여러 객체를 만들 수 있다. 각 객체는 같은 구조와 메서드를 공유하지만 인스턴스 변수는 별도로 저장한다.

```

1 student1 = Student("Kim")
2 student2 = Student("Lee")

```

메모리의 개념적인 구조는 다음과 같다.

```

student1
  name -> "Kim"

student2
  name -> "Lee"

```

6.1. 한 객체만 변경하기

```

1 student1.name = "Park"

```

이 코드는 student1의 속성만 변경한다. student2.name은 여전히 "Lee"이다.

```

1 student1.introduce()
2 student2.introduce()

```

실행 결과는 다음과 같다.

My name is Park.

My name is Lee.

6.2. 같은 클래스와 서로 다른 객체

구분	student1	student2
클래스	Student	Student
name	"Park"	"Lee"
사용 메서드	introduce()	introduce()

클래스의 메서드 정의는 공유하지만 객체마다 데이터는 독립적으로 유지된다. 이것이 하나의 설계도로 여러 실제 대상을 효율적으로 관리할 수 있는 이유이다.

6.3. 최종 코드: 05_multiple_objects.py

```

1 class Student:
2     def __init__(self, name):
3         self.name = name
4
5     def introduce(self):
6         print(f"My name is {self.name}.")
7
8
9 student1 = Student("Kim")
10 student2 = Student("Lee")
11
12 student1.introduce()
13 student2.introduce()
14
15 print()
16
17 student1.name = "Park"
18
19 student1.introduce()
20 student2.introduce()

```

실습

Car 클래스에 brand, year 속성과 show.info() 메서드를 만든다. 객체 두 개를 생성한 뒤 첫 번째 객체의 연도만 변경하여 두 객체가 독립적인지 확인한다.

7. Day 5 종합 실습: 학생 관리 프로그램

이번 종합 실습에서는 클래스, 생성자, 인스턴스 변수, 인스턴스 메서드, self, 여러 객체와 상태 변경을 모두 사용한다.

7.1. 프로그램 요구사항

1. Student 클래스에 name, age, major를 저장한다.
2. introduce()는 학생 정보를 출력한다.
3. birthday()는 나이를 1 증가시킨다.
4. change_major()는 전공을 새로운 값으로 변경한다.

5. 학생 객체 세 개를 생성한다.
6. 첫 번째 학생의 나이와 전공만 변경한다.
7. 모든 학생 정보를 다시 출력하여 객체의 독립성을 확인한다.

7.2. 최종 코드: practice_day05.py

```
1 class Student:
2     def __init__(self, name, age, major):
3         self.name = name
4         self.age = age
5         self.major = major
6
7     def introduce(self):
8         print(f"Name: {self.name}")
9         print(f"Age: {self.age}")
10        print(f"Major: {self.major}")
11
12    def birthday(self):
13        self.age += 1
14        print(f"Happy Birthday, {self.name}!")
15
16    def change_major(self, major):
17        self.major = major
18        print(f"Major changed to {self.major}")
19
20
21 st1 = Student("Kim", 20, "Chemistry")
22 st2 = Student("Lee", 21, "Mathematics")
23 st3 = Student("Park", 22, "Education")
24
25 st1.introduce()
26 st2.introduce()
27 st3.introduce()
28
29 print()
30
31 st1.birthday()
32 st1.change_major("Business")
33
34 print()
35
36 st1.introduce()
37 st2.introduce()
38 st3.introduce()
```

7.3. 실행 결과

```
Name: Kim
Age: 20
Major: Chemistry
Name: Lee
Age: 21
Major: Mathematics
Name: Park
Age: 22
```

Major: Education

Happy Birthday, Kim!

Major changed to Business

Name: Kim

Age: 21

Major: Business

Name: Lee

Age: 21

Major: Mathematics

Name: Park

Age: 22

Major: Education

7.4. 프로그램 구조 분석

기능	사용한 개념
학생 설계도 정의	클래스
초기 정보 저장	생성자와 인스턴스 변수
정보 출력	인스턴스 메서드
현재 객체 구분	<code>self</code>
나이 증가	객체 상태 변경
전공 변경	매개변수를 받는 메서드
학생 세 명 관리	여러 객체 생성
첫 번째 학생만 변경	객체의 독립성

핵심 포인트

`birthday()`와 `change_major()`는 외부에서 속성을 직접 수정하는 대신 객체 자신의 메서드로 상태를 변경한다. 객체가 자신의 데이터를 관리하게 만드는 것이 객체지향적인 설계의 기본 방향이다.

8. 실습 파일 목록

실습

Day 5의 학습 파일은 다음과 같이 관리한다.

- 01_first_class.py
- 02_constructor.py
- 03_instance_methods.py
- 04_self.py
- 05_multiple_objects.py
- practice_day05.py
- python_day05.tex
- python_day05.pdf

9. 핵심 요약

- 클래스는 객체를 만들기 위한 설계도이며 새로운 자료형을 정의한다.
- 객체 또는 인스턴스는 클래스로부터 생성된 실제 값이다.
- `class ClassName:` 형식으로 클래스를 정의한다.
- `ClassName()`을 호출하면 새로운 객체가 생성된다.
- `__init__()`는 객체 생성 직후 자동으로 호출되는 초기화 메서드이다.
- 생성자의 일반 매개변수는 전달받은 값이고, `self.name`은 객체에 저장되는 인스턴스 변수이다.
- 인스턴스 메서드는 객체의 속성을 읽거나 변경하는 동작을 정의한다.
- `self`는 현재 메서드를 호출한 객체 자신을 가리킨다.
- `object.method()` 호출에서 Python은 `object`를 `self`에 자동으로 전달한다.
- 하나의 클래스에서 여러 객체를 만들 수 있다.
- 같은 클래스에서 만든 객체들은 메서드 구조를 공유하지만 인스턴스 변수는 서로 독립적이다.
- 한 객체의 속성을 변경해도 다른 객체의 속성은 변경되지 않는다.
- 객체의 상태 변경을 메서드로 표현하면 데이터와 동작을 하나의 클래스 안에 묶을 수 있다.